Lecture 60 : Software Testing

Introduction

In the complex world of software development, delivering a product that consistently performs as expected is paramount. This lecture delves into the critical aspects of software testing, a discipline essential for achieving high-quality and reliable software. Building upon previous discussions on reliability estimation, we transition to understanding the practical challenges and methodologies involved in identifying and mitigating defects. From distinguishing between subtle errors and catastrophic failures to strategizing how and when to conclude testing, this session provides a foundational understanding for anyone involved in creating or managing software projects. The insights gained here are crucial for project managers, developers, and testers alike, highlighting why testing is not merely a routine task but a sophisticated and effort-intensive process that underpins the trustworthiness of any software application.

Core Concepts

Term	Definition	Significance
`Reliability Growth Modeling`	A method to estimate software reliability by tracking the failure rate over time and fitting it to a mathematical model.	Helps predict future reliability and assess the effectiveness of defect removal processes.
`Statistical Testing`	A reliability estimation technique that uses an operational profile to design test cases, focusing on how users typically interact with the software.	Provides a realistic estimate of reliability as perceived by average users by prioritizing frequently used functionalities.
`Operational Profile`	A description of the typical usage patterns of a software system, including the frequency of invoking different functions.	Essential for designing effective statistical tests that reflect real-world usage and for estimating user-perceived reliability.

Term	Definition	Significance
`Fault`	An incorrect step, process, or data definition in a computer program; also known as a defect or bug.	The underlying cause of a failure, which may or may not manifest as an observable problem depending on execution path.
`Failure`	The inability of a system or component to perform its required functions within specified performance requirements.	The observable manifestation of a fault during program execution, indicating a deviation from expected behavior.
`Error`	A human mistake or misconception that leads to a fault in the software.	The origin point of faults, often occurring during specification, design, or coding phases.
`Verification`	The process of evaluating whether the products of a given development phase satisfy the conditions imposed at the start of that phase.	Ensures that each stage of development correctly implements the requirements of the previous stage ("Are we building the product right?").
`Validation`	The process of evaluating software during or at the end of the development process to determine whether it satisfies specified requirements.	Confirms that the final system meets the user's needs and original specifications ("Are we building the right product?").
`Corner Cases`	Input values or combinations of inputs that are unlikely to be given by typical users but can expose critical system failures.	Crucial for thorough testing, as these edge conditions often trigger latent faults that normal usage might miss.

Detailed Analysis

1. Estimating Software Reliability

Software reliability can be estimated using two primary approaches: reliability growth modeling and statistical testing.

- **Reliability Growth Modeling**:
- Involves collecting failure data over time for a specific software application.
- Plots this data to observe the failure rate trend.
- Compares the observed trend against proposed reliability growth models (e.g., exponential, logarithmic) to find the most accurate fit.
- Allows for prediction of software reliability at present or in the future based on the chosen model.
- **Statistical Testing**:
- **Objective**: Primarily to estimate software reliability, not just to detect bugs.
- **Methodology**:
- Uses an `operational profile` to design test cases.
- Test data frequency corresponds to the frequency of function invocation by users (e.g., if a function is used 40% of the time, 40% of test data should target it).
- Relies on large test datasets to ensure statistical significance for reliable inference and prediction.
- Assumes that only a small percentage of test inputs are likely to cause system failure; if failures are rampant, reliability prediction becomes inaccurate.
- **Challenges**:
- **Test Data Generation**: Generating a statistically significant and large number of test data points is complex.
- **Corner Cases**: It's difficult to generate `unlikely input data` or `corner cases` which are crucial for uncovering hidden faults, as these are often the inputs that cause failures.
- **Operational Profile Definition**: The accuracy of results heavily depends on how precisely the operational profile is defined, often requiring laborious study of user logs.
- **Test Case Design**: Even with a correct operational profile, designing test cases that proportionally include unlikely inputs is a significant hurdle.
- **Advantages**:

- Tests frequently used parts of the software more thoroughly, leading to the removal of bugs in critical areas.
- Provides a more accurate estimation of reliability as perceived by the average user because test data is based on actual usage patterns.

2. Distinguishing Between Error, Fault, and Failure

Understanding the precise terminology is crucial in software testing.

- **Error**: A human mistake made during the software development process (e.g., in specification, design, or coding).
- **Fault (Bug/Defect)**: The manifestation of an error in the code or documentation. It's a flaw in the system that could lead to a failure. Not all faults cause failures; some may never be triggered.
- Example: A programmer writes `a = c` instead of `a = b`. If `b` and `c` happen to have the same value at runtime, this fault might not cause a failure.
- **Failure**: An observable event where the software deviates from its expected behavior. It is the manifestation of a fault during execution.
- When testing, testers observe failures (e.g., "fatal error," program termination).
- A single fault can cause multiple failures, and a failure can be caused by multiple faults.
- **Evolution of Definitions**:
- IEEE standard 1044 (1993) initially considered errors and faults as synonyms.
- The 2010 revision introduced finer distinctions to improve communication and expressiveness in the field.

3. Bug Distribution and Reduction Strategies

Bugs are an inevitable part of software development due to its manual nature.

- **Bug Prevalence**:
- Even proficient programmers make mistakes.
- Studies show an average of 50 bugs per 1000 lines of source code after initial programming.
- After thorough testing, typically 1 bug per 1000 lines of source code may still remain.

- This remaining bug density might be acceptable for some applications but is critical for safety-critical systems (e.g., banking, nuclear power plants).
- **Bug Distribution by Origin**:
- Majority of bugs (around 60%) originate from specification and design phases.
- Approximately 40% of bugs are due to coding mistakes.
- **Strategies for Bug Reduction**:
- **Review**: Systematically reviewing work at various stages (e.g., code reviews, design reviews).
- **Testing**: The most important and effective technique for identifying and removing bugs.
- **Formal Specification and Verification**: Using rigorous mathematical methods to define and verify software behavior.
- **Proper Development Process**: Adhering to well-defined software development methodologies to minimize errors.

4. The Software Testing Process

Testing involves a systematic approach to uncover failures.

- **Basic Process**:
- Provide input data to the software.
- Observe the output.
- Compare the observed output with the expected output for the given input.
- If the program does not behave as expected, a failure is identified.
- **Test Report**: When a failure occurs, a test report is created, documenting:
- The specific test data used.
- The conditions under which the test data was given (e.g., software state).
- The output produced.
- Whether it was a failure.
- **Developer Action**: Developers use test reports to reproduce failures, debug the code, and correct the software.

5. Effort and Time Allocation in Testing

Testing is a highly resource-intensive activity within software development.

- **Effort vs. Time**:

- Testing consumes the largest portion of development effort (around 50% for typical software).
- However, it often occupies a relatively small percentage of the total development time (e.g., 10%).
- **Parallelism**: This discrepancy is possible due to the high degree of parallelism in testing activities.
- Many testers can work simultaneously, testing different functionalities or using different test data.
- In contrast, activities like design and coding have less inherent parallelism, requiring more sequential effort from fewer individuals.
- **Job Opportunities**: The significant manpower required for testing translates into numerous job opportunities in the field.

6. Evolution and Challenges in Testing

The field of software testing is dynamic and increasingly complex.

- **Driving Factors**:
- Larger and more complex programs.
- Emergence of new programming paradigms.
- Advancements in test automation.
- Continuous research and new testing techniques.
- **Challenging Nature**: Testing is no longer considered a routine task; it requires sophisticated knowledge of techniques and tools. Early "monkey testing" (random inputs) has been replaced by more structured and intelligent approaches.

7. When to Stop Testing

Deciding when to conclude testing is a critical management decision.

- **Life Cycle Integration**: Testing is not a single phase but is carried out throughout the software development life cycle (e.g., unit testing, integration testing, system testing, usability testing).
- **Stopping Criteria**:
- **Bug Detection Rate**: Observe the number of bugs detected per unit of time. Testing may stop when the rate of new bug detection significantly decreases or reaches zero over a sustained period (e.g., a day or a week).
- **Bug Seeding**: A manager intentionally introduces known bugs into the program without the testers' knowledge. Testing stops when a high

percentage of these seeded bugs are detected, indicating the testing process is effective at finding defects.

8. Verification vs. Validation

These two terms represent distinct but complementary quality assurance activities.

- **Verification**:
- "Are we building the product right?"
- Determines if the output of one development phase conforms to the requirements of the previous phase.
- Examples: Checking if design results are consistent with specifications, or if code correctly implements the design.
- Performed iteratively as development proceeds.
- **Validation**:
- "Are we building the right product?"
- Determines if the fully developed system conforms to its overall specifications and meets user needs.
- Performed on the completed system or major components.

Key Takeaways

- [ ] Software reliability can be estimated using reliability growth modeling or statistical testing.
- [ ] Statistical testing uses an operational profile to design test cases, aiming to estimate user-perceived reliability.
- [ ] A `fault` is an underlying defect, while a `failure` is its observable manifestation during execution.
- [ ] `Errors` are human mistakes that lead to `faults`.
- [ ] Most bugs originate from specification and design, not just coding.
- [ ] Testing is the most effective technique for reducing bugs, consuming significant effort but often less time due to parallelism.
- [ ] The decision to stop testing can be based on the rate of bug detection or the success rate of finding seeded bugs.
- [ ] `Verification` ensures each development phase is done correctly, while `validation` confirms the final product meets overall requirements.
- [ ] `Corner cases` are crucial for thorough testing, as they often expose critical system failures.

Conclusion

This lecture has underscored the indispensable role of software testing in delivering reliable and high-quality products. From the nuanced distinction between errors, faults, and failures to the strategic deployment of statistical testing based on operational profiles, we've explored how rigorous testing practices are fundamental to software trustworthiness. The significant effort and evolving sophistication of testing methodologies reflect its critical importance in an era of increasingly complex software systems. Understanding these concepts empowers project managers to allocate resources effectively, developers to write more robust code, and testers to apply advanced techniques. Ultimately, a well-tested software application not only meets its functional requirements but also instills confidence in its users, a cornerstone of success in the digital landscape. As software continues to permeate every aspect of our lives, the principles of effective testing will remain at the forefront of engineering excellence.